Lab Session Tasks Week 6

Task 1: Find the minimum!

Create an algorithm to find the smallest entry in a list of items.

1. Define the problem precisely
   A. Do you need to consider any requirements?
   B. Could there be special cases?
2. Sketch an algorithm in pseudocode
3. Carefully check your algorithm with paper and pencil
   - Will it work correctly in all cases?
   - Hint: Specify an invariant for each loop
4. Analyse runtime: For a list of N elements, how many steps will your algorithm need?
5. Implement your algorithm in Python!

Problem definition

- **Input**: A sequence $A = \langle a_1, a_2, \ldots, a_n \rangle$ of $n \geq 1$ items for which a strict ordering is defined (i.e., $a_i < a_j$ is defined for all $1 \leq i, j \leq n$).
- **Output**: A value $v \in A$ such that $v \leq a_k$ for all $1 \leq k \leq n$.

Pseudocode

```
1: minimum(A)
2:     v = A[1]
3:     for j = 2..A.length
4:         if A[j] < v
5:             v = A[j]
6:     return v
```

Checking the algorithm

Invariant

The following invariant holds for the loop on lines 3–5:

> v holds the smallest value in $A[1..j-1]$.

Where exactly do we check the invariant?

The pseudocode notation `for j = 2..A.length` represents three different operations:

1. initialization of `j` with value 2 when we first encouter the loop
2. check of `j` against the upper bound `A.length` before we enter the
3. increase of `j` at the end of the loop body

Steps 2 and 3 are repeated for each round through the loop. This can be seen more easily by re-writing the for-loop as a while-loop

```
j = 2
while j ≤ A.length
   # do something
   j = j + 1
```

We test the invariant every time we perform the comparison `j ≤ A.length`, i.e., every time *after* j has received a new value, either by initialization or by increment.

Proving correctness

Initizlization

For $j = 2$ the invariant states that v holds the smallest value in $A[1..1] = A[1]$. This is true because we have initialized $v = A[1]$ on line 2.

Maintenance

We assume that the invariant holds up to $j = J$, i.e., that v is the smallest value in $A[1..J-1]$. Now, we execute the loop for $j = J$. Then, on line 4 we compare $A[J]$ to v. There are two possible outcomes: If $A[J]$ is smaller than v, we set v to $A[J]$ on line 5. Since v by assumption contained the smallest value in $A[1..J-1]$ before and the new value $v = A[J]$ is even smaller, v now holds the smallest value in $A[1..J]$. In the other case, we have that $A[J]$ is at least as large as v, i.e., $A[J]$ must be at least as large as the smallest value in $A[1..J-1]$. Thus, v by assumption must contain the smallest element in $A[1..J]$. So in both cases, we are certain that v contains the smallest element in $A[1..J]$ when we return to line 3. There, we increase j to $J+1$ and then the invariant reads v *holds the smallest value in* $A[1..J+1-1] = A[1..J]$, i.e., we proven that if the invariant held for $A[1..J-1]$ it also holds for $A[1..J]$, qed.

Termination

The loop terminates when $j \leq n$ no longer holds (with $n = A.\text{length}$), i.e., when $j = n+1$. The algorithm will terminate, because j is increased by 1 for each round through the loop on lines 3–5 and n is constant and finite. The invariant then states that v holds the smallest value in $A[1..n+1-1] = A[1..n] = A$, which is equivalent to $v < a_k \; \forall 1 \leq k \leq n$. Thus, our algorithm has given the desired result.

Runtime analysis

- Line 2 will be executed 1 time at cost c_2

- Line 3 will be executed n times at cost c_3 (including the one time after the last round through the loop)
- Line 4 will be executed $n-1$ times at cost c_4
- Line 5 will be executed $t_j \in \{0,1\}$ times at cost c_5, depending on the data
- Line 6 will be executed 1 time at cost c_6

We then have

$$\begin{aligned} T(n) &= c_2 + c_3 n + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \\ &= c_5 \sum_{j=2}^{n} t_j + (c_3 + c_4)n + (c_2 - c4) \\ &= A \sum_{j=2}^{n} t_j + Bn + C \ . \end{aligned}$$

Now, in the best case $A[1]$ contains the minimum and we never execute line 5, i.e., all $t_j = 0$, and in the worst case line 5 is executed every time, i.e., $t_j = 1$ for all j. Thus we have

$$\begin{aligned} T_{\text{best}} &= Bn + C \\ T_{\text{worst}} &= A(n-1) + Bn + C = (A+B)n + (C-A) \ . \end{aligned}$$

Thus, in both cases we have linear functions and $T(n) = \Theta(n)$.

Python implementation

In [1]:
```
def minimum(data):
    min_value = data[0]
    for next_value in data[1:]:
        if next_value < min_value:
            min_value = next_value
    return min_value
```

Apply to a list of numbers

In [2]:
```
minimum([2, 4, 1, 7])
```

Out[2]:
```
1
```

Apply to a list of strings

In [5]:
```
minimum(["Per", "Kari", "Joe"])
```

Out[5]:
```
'Joe'
```

Apply to an empty list

In [8]:
```
minimum([])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 minimum([])

Cell In[1], line 2, in minimum(data)
      1 def minimum(data):
----> 2     min_value = data[0]
      3     for next_value in data[1:]:
      4         if next_value < min_value:

IndexError: list index out of range
```

Error: Our algorithm isn't defined for empty lists.

Apply to a list of complex numbers

In [6]:
```
minimum([1+1j, 1-1j])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[6], line 1
----> 1 minimum([1+1j, 1-1j])

Cell In[1], line 4, in minimum(data)
      2 min_value = data[0]
      3 for next_value in data[1:]:
----> 4     if next_value < min_value:
      5         min_value = next_value
      6 return min_value

TypeError: '<' not supported between instances of 'complex' and 'complex'
```

Error: Ordering is not defined for complex numbers.

Task 2: Find the largest three elements

1. Precisely define the expected output and give an algorithm for the following problem: Given a list of at least three numbers, return the three largest numbers in the list.
2. Where in Norwegian life is this algorithm relevant (since 2022)?
3. Give an invariant for each loop in the algorithm.
4. Argue that your algorithm works correctly.
5. Analyze the runtime of the algorithm.

6. Suggest an approach to testing actual implementations of the algorithm.
7. Implement the algorithm in Python.

Problem definition

- **Input**: A sequence $A = \langle a_1, a_2, \ldots, a_n \rangle$ of $n \geq 3$ items for which a strict ordering is defined (i.e., $a_i < a_j$ is defined for all $1 \leq i, j \leq n$).
- **Output**: A sequence $V = \langle v_1, v_2, v_3 \rangle$ with $v_j \in A$ for $1 \leq j \leq 3$ such that $v_{\min} \geq a_k$ for all $1 \leq k \leq n$ where $v_{\min} = \min_j v_j$.

Note that the output definition does not require that V is sorted.

Real-life relevance

One component of the electricity price paid by Norwegian households is determined by finding the three hours during a month (out of around 720) during which electricity consumption was maximal and then take the average of these three values.

Pseudocode

One possible implementation is

```
1: top_three(A)
2:     V = A[1..3]
3:     sort3_descending(V)
4:     for j = 4..A.length
5:         if A[j] > V[3]
6:             V[3] = A[j]
7:             sort3_descending(V)
8: return V
```

Here we use

```
1: sort3_descending(V)
2:     if V[2] < V[1]
3:         swap(V[1], V[2])
4:     if V[3] < V[2]
5:         swap(V[2], V[3])
6:     if V[2] > V[1]
7:         swap(V[1], V[2])
```

Checking the algorithm

Sorting function

Assume we originally have $\langle v_1, v_2, v_3 \rangle = \langle a, b, c \rangle$. Then, if $a < b$, the first two elements trade places on lines 2–3, and v_2 will contain the smaller one of a, b when we arrive at

line 4. On lines 4–5, we swap v_2 with $v_3 = c$ if c is smaller than v_2, i.e., smaller than the minimum of a and b. So we are certain that v_3 contains the minimum of a, b, c when we get to line 6. We now compare the first two elements and ensure the larger of them, and therefore the maximum of a, b, c will be in v_1 once line 7 completes. Thus, we have the maximum in v_1 and the minimum in v_3 and so the sequence V is sorted in descending order.

Invariant

The following invariant holds for the loop on lines 3–7:

> V holds the smallest three largest values in $A[1..j-1]$ and $V[3]$ contains the smallest value in V.

Proving correctness

Initizlization

For $j = 4$ the invariant states that V holds the smallest values in $A[1..3]$ and that $V[3]$ contains the smallest value in $V[1..3]$. The first part holds because we have initialized $V = A[1..3]$ on line 2. The second part holds because we have sorted V in descending order on line 3 and we have shown the correctness of `sort3_descending` above.

Maintenance

We assume that the invariant holds up to $j = J$, i.e., that V contains the largest three value in $A[1..J-1]$ and that $V[3]$ contains the smallest value in V. Now, we execute the loop for $j = J$. Then, on line 4 we compare $A[J]$ to $V[3]$. There are two possible outcomes: If $A[J]$ is larger than $V[3]$, then $V[3]$ as the smallest element in V needs to leave V and $A[J]$ needs to be added to V. We achieve this by setting $V[3]$ to $A[J]$ on line 6. Since V by assumption contained the three largest values in $A[1..J-1]$ before and the new value $V[3] = A[J]$ is larger than the smallest of those three, V now holds the largest three values in $A[1..J]$. In the other case, we have that $A[J]$ is no larger than the smallest value in V and thus V already holds the three largest values in $A[1..J]$. So in both cases, we are certain that V contains the three largest elements in $A[1..J]$ when we return to line 4. There, we increase j to $J+1$ and then the first part of the invariant reads V *holds the three largest values in* $A[1..J+1-1] = A[1..J]$, i.e., we proven that if the invariant held for $A[1..J-1]$, the first part also holds for $A[1..J]$. Concerning the second part of the invariant, if the condition on line 5 is not fulfilled, we do not modify V and so the second part holds trivially when we return to line 4. If the condition is fulfilled, we modify V on line 6 and the sort V on line 7, whence $V[3]$ will contain the minimum of V when we return to line 4, as required. Thus, both parts of the invariant will be fulfilled also for $j = J+1$, qed.

Termination

The loop terminates when $j = n + 1$. The algorithm will terminate, because j is increased by 1 for each round through the loop on lines 3–5 and n is constant and finite. The invariant then states that V holds the three largest values in $A[1..n+1-1] = A[1..n] = A$ as desired. The second part of the invariant is not relevant here.

Runtime analysis

We first consider `sort3_descending`. Here, lines 2, 4 and 6 will always be executed once with cost s_1 each, while lines 3, 5 and 7 will each be executed zero or one times with cost s_2 each. The total cost will thus be between $3s_1$ and $3s_1 + 3s_2$.

For the main algorithm, we have

- Line 2 will be executed 1 time at cost c_2
- Line 3 will be executed 1 time at cost c_3 with $3s_1 \leq c_3 \leq 3s_1 + 3s_2$
- Line 4 will be executed $n - 2$ times at cost c_4
- Line 5 will be executed $n - 3$ times at cost c_5
- Line 6 will be executed $t_j \in \{0, 1\}$ times at cost c_6, depending on the data
- Line 7 will be executed $t_j \in \{0, 1\}$ times at cost $c_{7,j}$ with $3s_1 \leq c_{7,j} \leq 3s_1 + 3s_2$, depending on the data
- Line 8 will be executed 1 time at cost c_8

To simplify the further analysis and assume the worst case for the sortings, i.e., that we always execute all lines in the sorting algorithm, i.e., $c_3 = c_{7,j} = 3s_1 + 3s_2 = 3S$ for all j.

We then have

$$\begin{aligned} T(n) &= c_2 + c_3 n + c_4(n-2) + c_5(n-3) + (c_6 + c_7)\sum_{j=4}^{n} t_j + c_8 \\ &= (c_6 + 3S)\sum_{j=4}^{n} t_j + Bn + C \ . \end{aligned}$$

Now, in the best case $A[1..3]$ contains the three largest numbers, so we we never execute lines 6–7, i.e., all $t_j = 0$, and in the worst case lines 6–7 are executed every time, i.e., $t_j = 1$ for all j. Thus we have

$$\begin{aligned} T_{\text{best}} &= Bn + C \\ T_{\text{worst}} &= (c_6 + 3S)(n-4) + Bn + C = (c_6 + B + 3S)n + (C - 4(c_6 + 3S)) \ . \end{aligned}$$

Thus, in both cases we have linear functions and $T(n) = \Theta(n)$ as for the search for a single minimum.

Optimization

The algorithm as given will always keeps V sorted, which is not strictly required. As an additional exercise argue that the algorithm will also work correctly is we remove the last two lines from `sort3_descending` (which should be renamed then) and that the term $3S$ should then be replaced with $2S$ in the expression for $T(n)$.

Testing the algorithm

One approach to testing would be to create a sequence of numbers $\langle 1, 2, \ldots N \rangle$, present various random permutations of this list to the algorithm and ensure that it always returns the values $N-2, N-1, N$, in any order.

Python implementation

In [14]:
```
def sort3_descending(data):
    if data[1] > data[0]:
        data[0], data[1] = data[1], data[0]
    if data[2] > data[1]:
        data[1], data[2] = data[2], data[1]
    if data[1] > data[0]:
        data[0], data[1] = data[1], data[0]

def top_three(A):
    top = A[:3]
    sort3_descending(top)
    for v in A[3:]:
        if v > top[2]:
            top[2] = v
            sort3_descending(top)

    return top
```

Test on a manual test case

In [15]:
```
top_three([2, 4, 3, 1])
```

Out[15]:
```
[4, 3, 2]
```

Test on random permuations of a list

In [34]:
```
import random

l = list(range(10001))
te = list(l[-1:-4:-1])

for _ in range(1000):
    random.shuffle(l)
    assert top_three(l) == te
```

Measure runtime

In [39]:
```
%timeit top_three(l)
```

```
180 µs ± 2.71 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

Modified algorithm 1

In this variant, we remove the last two lines of the sorting algorithm. We only look at the Python implementation.

In [24]:
```
def put_min_at_end(data):
    if data[1] > data[0]:
        data[0], data[1] = data[1], data[0]
    if data[2] > data[1]:
        data[1], data[2] = data[2], data[1]

def top_three_mea(A):
    top = A[:3]
    put_min_at_end(top)
    for v in A[3:]:
        if v > top[2]:
            top[2] = v
            put_min_at_end(top)

    return top
```

Test on manual example

In [25]:
```
top_three_mea([2, 4, 3, 1])
```

Out[25]:
```
[4, 3, 2]
```

Test on random data

We are no longer guaranteed that the results of the algorithm are sorted, so we need to sort them before comparison to the expected values.

In [36]:
```
l = list(range(10001))
te = list(l[-1:-4:-1])

for _ in range(1000):
    random.shuffle(l)
    assert sorted(top_three_mea(l), reverse=True) == te
```

Measure runtime

In [38]:
```
%timeit top_three_mea(l)
```

```
174 µs ± 61.1 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

Modified algorithm 2

The first modified algorithm above saves a little work but gives us a slightly lesser result in the sense that the sequence V that is returned is no longer sorted.

Here, we consider a variant of the algorithm in which we maintain the sortedness of V without having to perform a full sort for each iteration. We start by sorting V before we enter the loop. Inside the loop, if we set a new $V[3]$, we compare it with $V[2]$ and swap if necessary and then see if we need to swap $V[1]$ and $V[2]$.

In [41]:
```
def top_three_rs(A):
    top = A[:3]
    sort3_descending(top)

    for v in A[3:]:
        if v > top[2]:
            top[2] = v
            if top[2] > top[1]:
                top[1], top[2] = top[2], top[1]
                if top[1] > top[0]:
                    top[0], top[1] = top[1], top[0]

    return top
```

Argue that this algorithm is also correct.

Test on manual example

In [42]:
```
top_three_rs([2, 4, 3, 1])
```

Out[42]:
```
[4, 3, 2]
```

Test on random data

Here, we again expect data in sorted order, so no sorting required.

In [43]:
```
l = list(range(10001))
te = list(l[-1:-4:-1])

for _ in range(1000):
    random.shuffle(l)
    assert top_three_rs(l) == te
```

Measure runtime

In [45]:
```
%timeit top_three_rs(l)
```

```
173 µs ± 45.1 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

Also here the runtime improvement is at best very small.

In [ ]: